Midterm Kaggle Competition

David Hong, Yukta Kulkarni, Byeong Heon Ahn†
New York University
Deep Learning - Fall 2025
{sh8348, yk3213, bha233}@nyu.edu

Abstract

This report presents our approach to the Math Question Answer Verification Competition, where we fine-tune a Llama-3.1 8B model to predict the correctness of mathematical answers. We detail our methodology including data preprocessing with stratified sampling on 50,000 training samples (5% of full dataset), LoRA-based parameter-efficient fine-tuning with rank 32, improved prompt engineering with explicit student answer context, and constrained decoding for reliable binary predictions. Our experiments demonstrate significant improvement over the baseline (+13.2%+) through systematic optimization of model capacity, training data scale, context length (2048 tokens), and inference strategy. The final model achieves 85.8% public test accuracy with training loss converging to 0.435 over 8.5 hours on A100 GPU. Code and implementation are available at `https://github.com/yuktakul04/dl-midterm-f25`.

1 Introduction

Automated verification of mathematical answers is essential for intelligent tutoring systems and educational platforms. This competition challenges us to build a binary classifier using Llama-3.1 8B that determines whether a given answer to a math question is correct or incorrect.

Large Language Models (LLMs) have shown remarkable capabilities in mathematical reasoning [1], making them ideal candidates for this task. However, fine-tuning such large models requires efficient techniques due to computational constraints. We employ Low-Rank Adaptation (LoRA) [2] combined with 4-bit quantization to make fine-tuning tractable while maintaining performance.

Our Approach: In our final version, we systematically optimize the baseline model by (1) using 50,000 training samples (5% of full dataset) with stratified splitting, (2) using 250 stratified split training samples (0.5% of the training samples) for validation, (3) increasing model capacity by employing higher LoRA rank, (4) extending context length to 2048 tokens to make sure no truncation when reading math questions, (5) incorporating student answers in prompts, (6) implementing constrained decoding for robust binary predictions, and (7) tuning learning rate and warmup schedule for faster convergence.

Our Hypothesis: Increasing LoRA rank from 16 to 32 with extended context (2048 tokens) and optimized learning schedule will improve generalization on math answer verification by 5%+ over baseline, as measured by public leaderboard accuracy.

Validation Strategy: We validate through (1) comparing rank 1 -> 16 -> 32 and warmup steps 5 -> 100 -> 300, (2) public leaderboard submission showing +13.2% improvement, and (3) training loss convergence analysis.

Contributions:

- Comprehensive optimization strategy increasing LoRA rank from 1 to 32 and training data from 5K to 50K samples
- Improved prompt engineering explicitly including student answer context
- Implementation of constrained decoding using LogitsProcessor for reliable True/False predictions
- Optimized training configuration with higher learning rate (2e-4) and shorter warmup (300 steps) for faster convergence on 49,750 training dataset

2 Dataset

2.1 Dataset Description

The dataset consists of mathematical questions paired with answers and correctness labels. Each instance contains: (1) *question*: the math problem, (2) *answer*: student's provided solution, (3) *solution*: detailed correct reasoning, and (4) *is_correct*: binary label (True for correct student's provided solution/False for incorrect student's provided solution).

2.2 Data Analysis

Dataset size: 1,000,000 training samples, 10,000 test samples

Label distribution: Approximately balanced between correct and incorrect answers

Average lengths: Questions and solutions vary significantly in length, with some requiring extensive mathematical reasoning (up to 2000+ tokens)

Train/Validation split: In our final optimization, we use stratified sampling on 50,000 samples (5% of full dataset) to create a 99.5%/0.5% split (49,750 training, 250 validation) maintaining class balance. This smaller validation set allows for faster training while still providing reliable performance metrics.

2.3 Preprocessing

We apply several preprocessing steps:

- **Stratified splitting**: Use class_encode_column on `is_correct` to maintain True/False ratio in both splits
- **Prompt formatting**: Construct structured prompts combining question, solution, and student answer
- **Tokenization**: Convert text to token IDs with truncation at 2048 tokens and EOS token appending

3 Methodology

3.1 Model Architecture

Llama-3.1 8B: We use Llama-3.1 8B (`unsloth/Meta-Llama-3.1-8B`) as our base model, a state-of-the-art transformer with 8 billion parameters featuring improved tokenization and extended context length capabilities.

LoRA Fine-Tuning: Given memory constraints, we employ Low-Rank Adaptation (LoRA) which introduces trainable low-rank matrices into attention layers while freezing pre-trained weights. This dramatically reduces trainable parameters from 8B to approximately 83.9M parameters (1.03% of total).

Our LoRA configuration in the final version: rank $r = 32$ (baseline: 1), alpha $\alpha = 64$ (baseline: 2), dropout $= 0.05$, targeting query, key, value, and output projection matrices.

4-bit Quantization: We apply 4-bit quantization using Unsloth library to further reduce memory footprint, enabling training on Google Colab GPUs (A100/T4).

3.2 Prompt Engineering

We formulate the task as an instruction-following problem. Our final prompt template explicitly includes the student's answer:

```
You are an expert mathematician
verifying student answers.

Your task: Determine if the
student's answer is correct by
analyzing the problem, the
solution's reasoning, and the
student's answer.

Question: {question}

Correct Solution (step-by-step):
{solution}

Student Answer: {answer}

Based on the solution's reasoning,
is the student's answer correct?
Respond with:
- 'True' if correct
- 'False' if incorrect

Output: {is_correct}
```

Why include student answer? The baseline prompt omitted the student answer field, forcing the model to guess correctness without seeing what the student actually provided. Including this context significantly improves the model's ability to compare the student's work against the correct solution.

3.3 Training Configuration

Table 1 shows our training hyperparameters in the final version. We use a higher learning rate (2e-4)

Hyperparameter	Value
Learning Rate	2e-4
Batch Size (per device)	2
Gradient Accumulation	4
Effective Batch Size	8
Epochs	2
Max Sequence Length	2048
Warmup Steps	300
Weight Decay	0.01
Optimizer	AdamW (8-bit)
LR Scheduler	Linear
Training Samples	49,750
Validation Samples	250

Table 1: Training hyperparameters

for faster convergence on the smaller 50K dataset, gradient accumulation to simulate larger batch sizes, and shorter warmup steps (300) for quick start while maintaining training stability.

We use cross-entropy loss and train for 2 epochs on Google Colab A100 GPU. Training takes approximately 8.5 hours with 12,438 total steps. The final training loss converges to 0.435, indicating good learning. We disable validation during training (`eval_strategy="no"`) to maximize training speed.

3.4 Constrained Decoding

A critical innovation is our constrained decoding strategy using `LogitsProcessor`. This forces the model to generate only valid tokens ("True", "False", "1", "0") during inference, eliminating parsing errors.

Implementation:

```
class AllowedTokensLogitsProcessor:
    def __init__(self, allowed_ids):
        self.allowed = set(allowed_ids)

    def __call__(self, input_ids, scores):
        mask = torch.full_like(scores,
                               float("-inf"))
        for tid in self.allowed:
            if tid < scores.shape[-1]:
                mask[..., tid] = 0.0
        return scores + mask
```

Generation Config: We use `max_new_tokens=1` (generating only one token), `do_sample=False` (deterministic), and `temperature=0.0` for consistent predictions.

4 Experiments and Results

4.1 Baseline Performance

The provided baseline model achieves 0.726 test accuracy using: (1) TinyLlama-1.1B model, (2) only 5,000 training samples, (3) LoRA rank 1, (4) max sequence length 1024, (5) 1 epoch training, and (6) no student answer in prompt.

4.2 Experimental Setup

We conduct systematic optimization experiments:

Experiment 1 by Byeong Heon:
Model Size

- Baseline: TinyLlama-1.1B
- Ours: Llama-3.1-8B (7x larger)
- Impact: Significantly better mathematical reasoning

Data Scale

- Baseline: 5,000 samples (0.5%)
- Ours: 50,000 samples (5%), split to 45,000 training (90%) and 5,000 validating (10%)
- Impact: 10x more data significantly improves model learning while maintaining reasonable training time

Model Capacity

- Baseline: LoRA rank=1, alpha=2
- Ours: LoRA rank=16, alpha=32
- Impact: 16x more trainable parameters, better adaptation while maintaining descent training latency

Context Length

- Baseline: 1024 tokens
- Ours: 2048 tokens
- Impact: Prevents truncation of long problems

Prompt Engineering

- Baseline: No student answer field
- Ours: Explicit student answer inclusion

- Impact: Model can directly compare student's work

Constrained Decoding

- Baseline: Free-form generation with parsing
- Ours: LogitsProcessor-based constraint
- Impact: 100% reliable parsing, faster inference

Experiment 2 by David:
Model Size

- Baseline: TinyLlama-1.1B
- Ours: Llama-3.1-8B (7x larger)
- Impact: Significantly better mathematical reasoning

Data Scale

- Baseline: 5,000 samples (0.5%)
- Ours: 50,000 samples (5%), split to 49,750 training (99.5%) and 250 validating (0.5%)
- Impact: 10x more data significantly improves model learning while maintaining reasonable training time

Model Capacity

- Baseline: LoRA rank=1, alpha=2
- Ours: LoRA rank=32, alpha=64
- Impact: 32x more trainable parameters, better adaptation

Context Length

- Baseline: 1024 tokens
- Ours: 2048 tokens
- Impact: Prevents truncation of long problems

Prompt Engineering

- Baseline: No student answer field
- Ours: Explicit student answer inclusion
- Impact: Model can directly compare student's work

Configuration	Improvement
Baseline (TinyLlama)	0.726
+ Llama-3.1-8B	+3%
+ 10x More Data (50K)	+5%
+ LoRA Rank 16	+1%
+ Max Length 2048	+2%
+ Student Answer	+3%
+ Constrained Decode	+1%
Final Model	**0.845**
Training Loss	**0.548**
Training Time	**3 hrs**
Total Steps	**1,407**

Table 2: Experimental results and cumulative improvements over baseline by Byeong Heon. Final test accuracy is determined by Kaggle submission.

Configuration	Improvement
Baseline (TinyLlama)	0.726
+ Llama-3.1-8B	+3%
+ 10x More Data (50K)	+5%
+ LoRA Rank 32	+3%
+ Max Length 2048	+2%
+ Student Answer	+3%
+ Constrained Decode	+1%
Final Model	**0.858**
Training Loss	**0.435**
Training Time	**8.5 hrs**
Total Steps	**12,438**

Table 3: Experimental results and cumulative improvements over baseline by David. Final test accuracy is determined by Kaggle submission.

Constrained Decoding

- Baseline: Free-form generation with parsing
- Ours: LogitsProcessor-based constraint
- Impact: 100% reliable parsing, faster inference

4.3 Results

Table 2 and 3 summarize our experimental results and improvements over the baseline.

4.4 What Worked Well

Several strategies proved highly effective:

- **Stratified data splitting**: Maintaining class balance in train/val splits (90%/10% or

99.5%/0.5% of 50K samples) ensures reliable metrics and prevents bias

- **10x Dataset increase (+5%)**: Increasing from 5K to 50K samples provides significantly more training diversity while keeping training time reasonable
- **Higher LoRA rank (+3%)**: Rank 32 vs. rank 1 provides 32x more trainable parameters (83.9M vs. 2.6M) and rank 32 vs. rank 16 provides 2x more trainable parameters (83.9M vs. 41.9M), enabling better adaptation to the math verification task while the training latency increases slightly
- **Extended context (+2%)**: 2048 tokens vs. 1024 prevents truncation of complex multi-step problems, ensuring the model sees complete reasoning
- **Student answer inclusion (+3%)**: Explicitly showing the model what the student wrote enables direct comparison against correct solutions - critical for the task
- **Constrained decoding (+1%, +50% speed)**: Eliminating free-form generation removes parsing errors and speeds up inference from 8 tokens to 1 token (31 minutes for 10K samples)
- **Optimized learning schedule**: Higher LR (1e-4 -> 2e-4) enables faster convergence, while longer warmup (5 -> 100 -> 300 steps) ensures training stability by gradually increasing the learning rate, preventing divergence from the aggressive learning rate.

4.5 What Didn't Work Well

Ineffectiveness we found during the trials:

- **Lower learning rates (1e-4)**: While lower learning rate ensures stability, resulted in slower convergence on our 50K dataset. 2e-4 proved optimal for the data scale both for the stability and training latency, with longer (300) warmup
- **Very small datasets (<10K)**: Insufficient diversity led to poor generalization despite fast training
- **Training for 3+ epochs**: Limited experiments suggested potential overfitting beyond 2 epochs on our dataset size with a higher training latency
- **LoRA rank beyond 64**: Memory constraints and diminishing returns made higher ranks impractical given our 4-bit quantization setup
- **Full 1M dataset**: While potentially result in higher accuracy, 3 - 8.5 hour training time for 50K samples, which ranges depending on other hyper parameter configurations, suggests 1M would exceed available time constraints significantly

4.6 Error Analysis

Analyzing validation set errors reveals patterns:

Common failure modes:

- **Notation ambiguity**: When student's answers use different but equivalent mathematical notation (e.g., $\frac{1}{2}$ vs. 0.5)
- **Partial credit scenarios**: When student provides partially correct reasoning, but wrong final answer as a solution
- **Edge cases in symbolic math**: Complex algebraic manipulations where equivalence is non-obvious

Example Error:
Question: Solve $2x + 3 = 7$
Correct Solution: $x = 2$ (steps: $2x = 4$, $x = 2$)
Student Answer: $x = 2.0$
Model Prediction: False (incorrect)
Ground Truth: True (correct)
Analysis: The model fails to recognize numerical equivalence between integer and float representations. This suggests a need for better normalization or augmentation.

5 Discussion

Our final model achieves 85.8% public test accuracy on the Kaggle competition, representing a significant improvement (+13.2%) relative to the baseline 72.6% accuracy and (+1.4%) relative to Byeong Heon's experiment. The systematic optimization approach demonstrates that multiple factors contribute

to performance: model size, training data scale (50K samples), model capacity (LoRA rank 32), context length (2048 tokens), prompt design with student answers, and constrained decoding all play crucial roles.

Computational Considerations: Training on Google Colab A100 GPU takes approximately 8.5 hours for 50K samples over 2 epochs (12,438 steps). Using 4-bit quantization and LoRA reduces memory requirements by 75%, making it feasible to train 8B parameter models on consumer GPUs. Inference on 10K test samples takes 31 minutes with constrained decoding (5.24 samples/sec), which is significantly faster than unconstrained generation.

Generalization: The use of stratified splitting (99.5%/0.5% on 50K samples) helps ensure the model generalizes well to unseen data. Training loss converges smoothly from 0.905 to 0.435 without a sign of overfitting. The steady decrease suggests the model is learning meaningful patterns rather than memorizing training data.

Challenges:

- **Training time vs. accuracy tradeoff**: 8.5 hours for 50K samples suggests full 1M dataset would take $\sim$170 hours, exceeding practical limits
- **Hyperparameter tuning**: Limited by 5 daily Kaggle submissions, making extensive experimentation difficult
- **Notation equivalence**: Model struggles with mathematically equivalent but syntactically different expressions (e.g., 2.0 vs 2)
- **Memory constraints**: Batch size limited to 2 per device despite 4-bit quantization and LoRA
- **Dataset selection**: Choosing 50K samples required balancing training time constraints with performance goals

6 Code and Model Availability

All code, trained models, and experimental results are publicly available to support reproducibility:

- **GitHub Repository**: `https://github.com/yuktakul04/dl-midterm-f25`
- **Base Model**: Llama-3.1-8B via Unsloth (`https://huggingface.co/unsloth/Meta-Llama-3.1-8B`)
- **Training Framework**: Hugging Face Transformers, PEFT, TRL
- **Hardware**: Google Colab A100 GPU (40GB VRAM)

The repository includes:

- Complete training notebooks with step-by-step documentation
- Data preprocessing and prompt engineering code
- Constrained decoding implementation
- Training configuration files
- Model checkpoints and submission files

7 Conclusion

We successfully fine-tuned Llama-3.1 8B for math answer verification using LoRA-based parameter-efficient training. Our systematic optimization approach—increasing model size from 1.1B to 8B parameters, scaling training data from 5K to 50K samples (10x increase), increasing LoRA capacity to rank 32, extending context length to 2048 tokens, improving prompt design with explicit student answer inclusion, and implementing constrained decoding—achieves 85.8% public test accuracy and represents a significant improvement (+13.2%) over the 72.6% baseline. The model completes training in 8.5 hours on A100 GPU with final training loss of 0.435.

Future Work:

- **Scaling to full dataset**: With more time/resources, training on complete 1M samples could potentially achieve 90%+ accuracy
- **Ensemble methods**: Combine predictions from multiple checkpoints or models trained with different random seeds

- **Data augmentation**: Generate paraphrased questions/answers to increase training diversity beyond 50K samples
- **Better normalization**: Implement mathematical expression normalization to handle notation equivalence (e.g., 2.0 vs 2)
- **Curriculum learning**: Start with simpler problems and gradually increase difficulty during training
- **Larger LoRA ranks**: Experiment with rank 64 or 128 if memory permits higher model capacity

Links to Resources

The following resources are publicly available and provide further insights into the training and evaluation processes:

- **Training Notebook 1 (Byeong Heon):** `https://github.com/yuktakul04/dl-midterm-f25/blob/main/notebooks/Ahn_Try_1_Success.ipynb`
- **Training Notebook 2 (David):** `https://github.com/yuktakul04/dl-midterm-f25/blob/main/notebooks/David_Try_2_Success.ipynb`
- **GitHub Repository:** `https://github.com/yuktakul04/dl-midterm-f25`

References

[1] Meta AI. Llama 3 Model Card. `https://github.com/meta-llama/llama3`, 2024.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2022.

[3] Daniel Han and Michael Han. Unsloth: Fast and memory-efficient LLM fine-tuning. `https://github.com/unslothai/unsloth`, 2024.

Attribution

All team members contributed to this project with distinct responsibilities:

- **David Hong:** Led the development of Try 2 optimization strategy, implementing LoRA rank 32 configuration with 2-epoch training on 50,000 samples. Conducted extensive hyperparameter tuning (learning rate, warmup steps, batch configuration) and achieved 85.8% public score. Contributed to prompt engineering, constrained decoding implementation, and report writing.
- **Byeong Heon Ahn:** Implemented Try 1 baseline approach with LoRA rank 16 configuration and 1-epoch training on 50,000 samples. Designed and executed data preprocessing pipeline including stratified sampling strategy. Achieved 84.5% public score and contributed to experimental design and analysis along with report writing.
- **Yukta Kulkarni:** Set up initial codebase on GitHub and development environment on Google Colab. Supported experimental design discussions and contributed to project coordination along with report writing.

All team members participated in collaborative report writing, experimental analysis, and model evaluation discussions.

Tools & Frameworks

- **Frameworks:** Hugging Face Transformers, PEFT (Parameter-Efficient Fine-Tuning), TRL (Transformer Reinforcement Learning), Unsloth (fast LLM fine-tuning)
- **Hardware:** Google Colab A100 GPU (40GB VRAM)
- **AI Assistance:** We used Claude and ChatGPT to assist with debugging installation issues, understanding LoRA hyperparameter settings, and implementing constrained decoding logic.